TECHNOLOGIA 2.0

Round 2 · Operation Phoenix

PARTICIPANT REFERENCE

Glossary & Hints

This booklet is a reference, not a rulebook. Use the glossary to look up unfamiliar terms from the Technology Inventory, and the hints to check your thinking — none of it hands you an answer.

Glossary

Plain-language definitions for terms you'll encounter across the Technology Inventory and Engineering Notes.

Term	What it means
SPA (Single Page App)	A frontend built as one page that updates its content dynamically, instead of reloading a new page for every screen.
CDN (Content Delivery Network)	A network of servers spread across locations that cache and serve content closer to the user, reducing load time.
SSR (Server-Side Rendering)	Generating a webpage's HTML on the server before sending it to the browser, rather than building it in the browser.
Monolithic Architecture	A design where the UI, business logic, and other concerns run together as one deployable unit, rather than as separate services.
Auto-Scaling Compute	Backend compute capacity (VMs or containers) that automatically increases or decreases based on traffic load.
Kubernetes	A system for running and managing many containers across machines, handling scheduling, scaling, and recovery automatically.
Serverless Functions (FaaS)	Small backend functions that run only when triggered and scale automatically, without managing servers directly.
Cold Start	The delay before a serverless function responds when it has to start up from an idle state.
Relational Database (RDBMS)	A structured database organized into tables with defined relationships, typically queried using SQL.
Read Replica	A copy of a database used to handle read queries, so the primary database is freed up mainly for writes.
Multi-AZ (Multi Availability Zone)	Running infrastructure across more than one physically separate data center location, so a failure in one doesn't take the system down.
NoSQL Document Store	A database that stores data as flexible, schema-less documents rather than fixed rows and columns.

Term	What it means
OAuth2 / OIDC	Standard protocols that let an app confirm a user's identity and permissions, often via a third-party identity provider, without the app storing passwords itself.
JWT (JSON Web Token)	A compact, signed token used to represent a user's identity and permissions between requests.
MFA (Multi-Factor Authentication)	Requiring more than one form of proof of identity to log in, such as a password plus a one-time code.
Message Queue	A component that holds messages between systems so tasks can be processed asynchronously instead of immediately, blocking the user.
Kafka-class Event Stream	A high-throughput system for publishing and consuming continuous streams of events, used when many services need to react to the same events in real time.
AI/ML Recommendation API	A ready-made, hosted service that returns personalized suggestions (e.g. products) without the team having to train or host a model.
GPU Instance	A server equipped with graphics processing hardware, used to train or run demanding machine-learning models faster.
PCI-DSS	A security standard that any system handling card payment data must follow to stay compliant.
APM (Application Performance Monitoring)	Tooling that tracks how an application is performing in production — response times, errors, and traces through the system.
In-Memory Cache (Redis / Memcached)	A fast, temporary data store kept in memory, used to avoid repeating expensive database reads for the same data.
Object Storage	Storage designed for large amounts of unstructured files (images, videos, backups), usually paired with a CDN for fast delivery.
RTO (Recovery Time Objective)	The maximum acceptable time to restore a system after a failure, before it's considered too slow.
Active-Active Deployment	Running full, independent copies of a system in more than one region at the same time, so both can serve live traffic.
Budget Unit / Cost Cap	The abstracted monthly cost figure assigned to each technology option in the simulation, used to check choices against the client's spending limit.

Hints

General hints

- There is no single correct architecture. The jury is evaluating your reasoning and clarity, not whether you matched a hidden answer key.
- Re-read the client requirements and business constraints before opening the Technology Inventory — it's easy to pick a popular-sounding option that doesn't actually fit the brief.

- The 60 minutes are split across four missions. Pace yourself; don't spend so long perfecting Mission 1 that later missions get rushed.
- Every deliverable asks for a short written justification somewhere — keep a running note of *why* you chose each component, in plain language.
- Cost and complexity are trade-offs, not penalties. A more expensive or more complex option can be the right call if the requirements justify it — say so.

Mission 1 – Project Discovery

- Separate the client's functional requirements (what the platform must do) from the business constraints (budget, team size, compliance, timeline) — you'll need both to justify a choice.
- For every required category, ask: does this option fit the budget cap, the team's ability to operate it, and the compliance needs — not just “which one sounds more powerful”?
- A smaller, simpler option is a legitimate choice for a small team on a tight timeline. Bigger isn't automatically better here.

Mission 2 – Architecture Foundation

- Trace one real user action (e.g. “a buyer places an order”) through your components, in order. Let that path shape your layout.
- Every connection you draw should represent an actual interaction — avoid connecting components just to fill space.
- Simplicity reads as clarity to a jury. A clean, well-labelled diagram beats a busy one.

Mission 3 – System Expansion

- Match each new requirement to the smallest architectural change that satisfies it — not every new requirement needs a brand-new tier.
- It's fine to revisit a Mission 1 or 2 choice if a new requirement genuinely demands it — just be ready to explain why.
- Keep a running total of cost as you add components; expansion is the stage where teams most often go over the budget cap.

Mission 4 – Final Integration

- Step back and check the whole diagram tells one coherent story before you touch anything else.
- A deployment plan doesn't need to be long — a few concrete stages (e.g. stage, verify, release, rollback trigger) are enough.
- Leave a few minutes at the end purely to check your final cost against the budget cap — it's an easy, avoidable mistake to submit over budget.